

Programación de aula basada en proyecto.
Desarrollo del backend de un pastillero inteligente.

VigiaSenior

Trabajo Realizado por:
María de los Ángeles Rodríguez Sánchez
Israel Cortés Gisbert

Didáctica II de la Tecnología y la Informática

Índice

1. Contexto Académico
2. Marco Legal y Referencias Normativas
3. Módulo Profesional
4. Proyecto VigiaSenior

Contexto Académico

- Etapa: Ciclo Formativo de Grado Superior
- Familia profesional: Informática y Comunicaciones
- Título: **Técnico Superior en Desarrollo de Aplicaciones Web (DAW)**
- Módulo: **0613 - Desarrollo web en entorno servidor**
- Curso: Segundo
- Horas : 200h / 6h semanales

Marco Legal y Referencias Normativas

Normativa Básica

- LOE (Ley Orgánica 2/2006), que se refiere a la FP en sus artículos 39 a 44. Está modificada por la LOMLOE (Ley Orgánica 3/2020).
- Ordenación e integración de la FP (Ley Orgánica 3/2022).
- Ordenación del sistema de FP (Real Decreto 659/2023).

Título y Currículo

- Título de Técnico Superior en Desarrollo de Aplicaciones Web (Real Decreto 686/2010). Modificado por “Actualización de Título de Técnico Superior en DAM y DAW” (Real Decreto 405/2023).
- Currículos de los ciclos formativos de grado medio y superior de Formación Profesional (Decreto 114/2025).
- Currículo del Título de Técnico Superior en DAW (Orden EDU/2887/2010).
- Regulación de la evaluación del proceso de enseñanza-aprendizaje en ciclos formativos (Orden 8/2025).
- Instrucciones sobre ordenación académica y de organización de los centros (Resolución de 17 de julio de 2025).

Módulo 0613 – objetivos generales

- c) Instalar módulos analizando su estructura y funcionalidad para gestionar servidores de aplicaciones.
- d) Ajustar parámetros analizando la configuración para gestionar servidores de aplicaciones.
- f) Seleccionar lenguajes, objetos y herramientas, interpretando las especificaciones para desarrollar aplicaciones web con acceso a bases de datos.
- g) Utilizar lenguajes, objetos y herramientas, interpretando las especificaciones para desarrollar aplicaciones web con acceso a bases de datos.
- h) Generar componentes de acceso a datos, cumpliendo las especificaciones, para integrar contenidos en la lógica de una aplicación web.
- l) Utilizar herramientas y lenguajes específicos, cumpliendo las especificaciones, para desarrollar e integrar componentes software en el entorno del servidor web.
- m) Emplear herramientas específicas, integrando la funcionalidad entre aplicaciones, para desarrollar servicios empleables en aplicaciones web.
- n) Evaluar servicios distribuidos ya desarrollados, verificando sus prestaciones y funcionalidad, para integrar servicios distribuidos en una aplicación web.
- ñ) Verificar los componentes de software desarrollados, analizando las especificaciones, para completar el plan de pruebas. o) Utilizar herramientas específicas, cumpliendo los estándares establecidos, para elaborar y mantener la documentación de los procesos.
- q) Programar y realizar actividades para gestionar el mantenimiento de los recursos informáticos.
- s) Desarrollar la creatividad y el espíritu de innovación para responder a los retos que se presentan en los procesos y organización de trabajo y de la vida personal.
- t) Tomar decisiones de forma fundamentada analizando las variables implicadas, integrando saberes de distinto ámbito y aceptando los riesgos y la posibilidad de equivocación en las mismas, para afrontar y resolver distintas situaciones, problemas o contingencias.

Módulo 0613 – competencias profesionales, personales y sociales

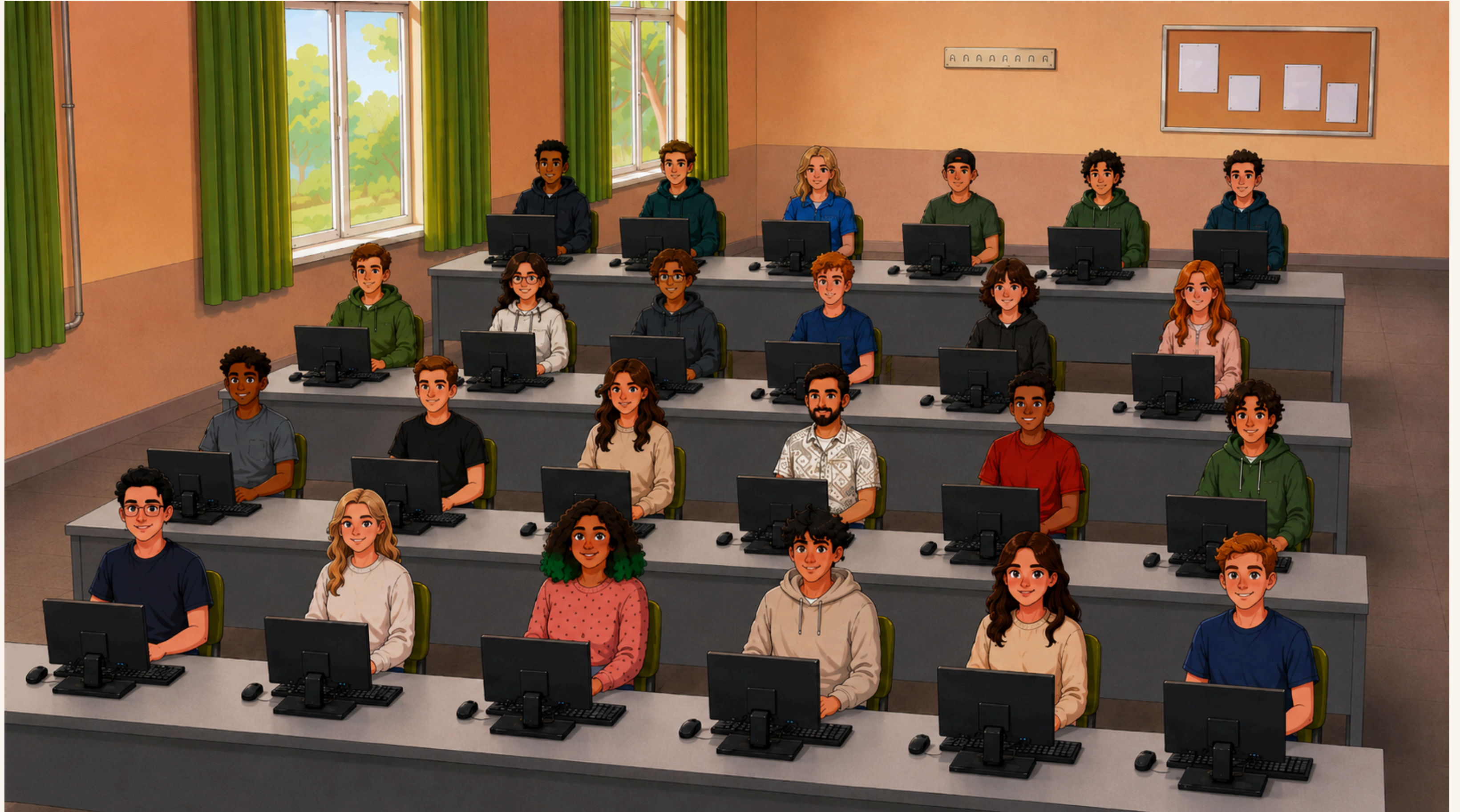
- c) Gestionar servidores de aplicaciones adaptando su configuración en cada caso para permitir el despliegue de aplicaciones web.
- d) Gestionar bases de datos, interpretando su diseño lógico y verificando integridad, consistencia, seguridad y accesibilidad de los datos.
- f) Integrar contenidos en la lógica de una aplicación web, desarrollando componentes de acceso a datos adecuados a las especificaciones.
- g) Desarrollar interfaces en aplicaciones web de acuerdo con un manual de estilo, utilizando lenguajes de marcas y estándares web.
- h) Desarrollar componentes multimedia para su integración en aplicaciones web, empleando herramientas específicas y siguiendo las especificaciones establecidas.
- k) Desarrollar servicios para integrar sus funciones en otras aplicaciones web, asegurando su funcionalidad.
- l) Integrar servicios y contenidos distribuidos en aplicaciones web, asegurando su funcionalidad.
- m) Completar planes de pruebas verificando el funcionamiento de los componentes software desarrollados, según las especificaciones.
- n) Elaborar y mantener la documentación de los procesos de desarrollo, utilizando herramientas de generación de documentación y control de versiones.
- ñ) Desplegar y distribuir aplicaciones web en distintos ámbitos de implantación, verificando su comportamiento y realizando modificaciones.
- q) Resolver situaciones, problemas o contingencias con iniciativa y autonomía en el ámbito de su competencia, con creatividad, innovación y espíritu de mejora en el trabajo personal y en el de los miembros del equipo.

Módulo 0613 –Bloques de Unidades de Programación y RA

Bloque	Unidad de Programación
Bloque 1. Introducción	• Selección de arquitecturas y herramientas de programación (RA1) • Inserción de código en páginas web (RA2) • Programación basada en lenguajes de marcas con código embebido (RA3)
Bloque 2. Programación en Entorno Servidor	• Desarrollo de aplicaciones Web utilizando código embebido (RA4) • Generación dinámica de páginas Web (RA5) • Utilización de técnicas de acceso a datos (RA6)
Bloque 3. Servicios Web y Aplicaciones Web Híbridas	• Programación de servicios Web (RA7) • Generación dinámica de páginas web interactivas (RA8) • Desarrollo de aplicaciones Web híbridas (RA9)



Módulo 0613 – Aula



Proyecto – RA/CCEE/Contenidos

Resultado de Aprendizaje	Criterios de Evaluación	Contenidos
RA5. Desarrolla aplicaciones web identificando y aplicando mecanismos para separar el código de presentación de la lógica de negocio.	a) Se han identificado las ventajas de separar la lógica de negocio de los aspectos de presentación de la aplicación. b) Se han analizado tecnologías y mecanismos que permiten realizar esta separación y sus características principales. e) Se han identificado y aplicado los parámetros relativos a la configuración de la aplicación Web. f) Se han escrito aplicaciones Web con mantenimiento de estado y separación de la lógica de negocio. g) Se han aplicado los principios de la programación orientada a objetos. h) Se ha probado y documentado el código.	Mecanismos de separación de la lógica de negocio. Controles de servidor. Mecanismos de generación dinámica del interface Web.
RA6. Desarrolla aplicaciones de acceso a almacenes de datos, aplicando medidas para mantener la seguridad y la integridad de la información.	a) Se han analizado las tecnologías que permiten el acceso mediante programación a la información disponible en almacenes de datos. b) Se han creado aplicaciones que establezcan conexiones con bases de datos. c) Se ha recuperado información almacenada en bases de datos. d) Se ha publicado en aplicaciones web la información recuperada. f) Se han creado aplicaciones web que permitan la actualización y la eliminación de información disponible en una base de datos. g) Se han utilizado transacciones para mantener la consistencia de la información. h) Se han probado y documentado las aplicaciones.	Establecimiento de conexiones. Recuperación y edición de información. Utilización de conjuntos de resultados. Ejecución de sentencias SQL. Transacciones. Utilización de otros orígenes de datos.

Proyecto – RA/CCEE/Contenidos

Resultado de Aprendizaje	Criterios de Evaluación	Contenidos
RA7. Desarrolla servicios web analizando su funcionamiento e implantando la estructura de sus componentes.	a) Se han reconocido las características propias y el ámbito de aplicación de los servicios Web. b) Se han reconocido las ventajas de utilizar servicios Web para proporcionar acceso a funcionalidades incorporadas a la lógica de negocio de una aplicación. c) Se han identificado las tecnologías y los protocolos implicados en la publicación y utilización de servicios Web. d) Se ha programado un servicio Web. e) Se ha creado el documento de descripción del servicio Web. f) Se ha verificado el funcionamiento del servicio Web. g) Se ha consumido el servicio Web.	Mecanismos y protocolos implicados. Generación de un servicio Web. Descripción del servicio. Interface de un servicio Web.
RA8. Genera páginas web dinámicas analizando y utilizando tecnologías del servidor web que añadan código al lenguaje de marcas.	c) Se han identificado las librerías y las tecnologías relacionadas con la generación por parte del servidor de páginas Web con guiones embebidos. d) Se han utilizado estas tecnologías para generar páginas Web que incluyan interacción con el usuario en forma de advertencias y peticiones de confirmación. e) Se han utilizado estas tecnologías, para generar páginas Web que incluyan verificación de formularios. f) Se han utilizado estas tecnologías para generar páginas web que incluyan modificación dinámica de su contenido y su estructura. g) Se han aplicado estas tecnologías en la programación de aplicaciones Web.	Librerías y tecnologías relacionadas. Generación dinámica de páginas interactivas. Obtención remota de información. Modificación de la estructura de la página Web.

Proyecto – RA/CCEE/Contenidos

Resultado de Aprendizaje	Criterios de Evaluación	Contenidos
RA9. Desarrolla aplicaciones web híbridas seleccionando y utilizando librerías de código y repositorios heterogéneos de información.	a) Se han reconocido las ventajas que proporciona la reutilización de código y el aprovechamiento de información ya existente. b) Se han identificado librerías de código y tecnologías aplicables en la creación de aplicaciones web híbridas. c) Se ha creado una aplicación web que recupere y procese repositorios de información ya existentes. d) Se han creado repositorios específicos a partir de información existente en Internet y en almacenes de información. e) Se han utilizado librerías de código para incorporar funcionalidades específicas a una aplicación web. f) Se han programado servicios y aplicaciones web utilizando como base información y código generados por terceros. g) Se han probado, depurado y documentado las aplicaciones generadas.	Reutilización de código e información. Utilización de información proveniente de repositorios. Incorporación de funcionalidades específicas.

- **Aprendizaje Basado en el Servicio** porque integra aprendizaje académico con servicio comunitario.
- No se hacen todas las fases. Se parte de que ya tenemos recogidas las necesidades del cliente del proyecto (la residencia de mayores).
- Desarrollo incremental por etapas y con entregables parciales.

Proyecto – RA

RA evaluados en la unidad de programación

- RA5 · Desarrolla aplicaciones web identificando y aplicando mecanismos para separar el código de presentación de la lógica de negocio.
- RA6 · Desarrolla aplicaciones de acceso a almacenes de datos, aplicando medidas para mantener la seguridad y la integridad de la información.
- RA7 · Desarrolla servicios web analizando su funcionamiento e implantando la estructura de sus componentes.

RA relacionados, no ponderados directamente:

- RA8 · Genera páginas web dinámicas analizando y utilizando tecnologías del servidor web que añadan código al lenguaje de marcas.
- RA9 · Desarrolla aplicaciones web híbridas seleccionando y utilizando librerías de código y repositorios heterogéneos de información.

Proyecto – Metodología

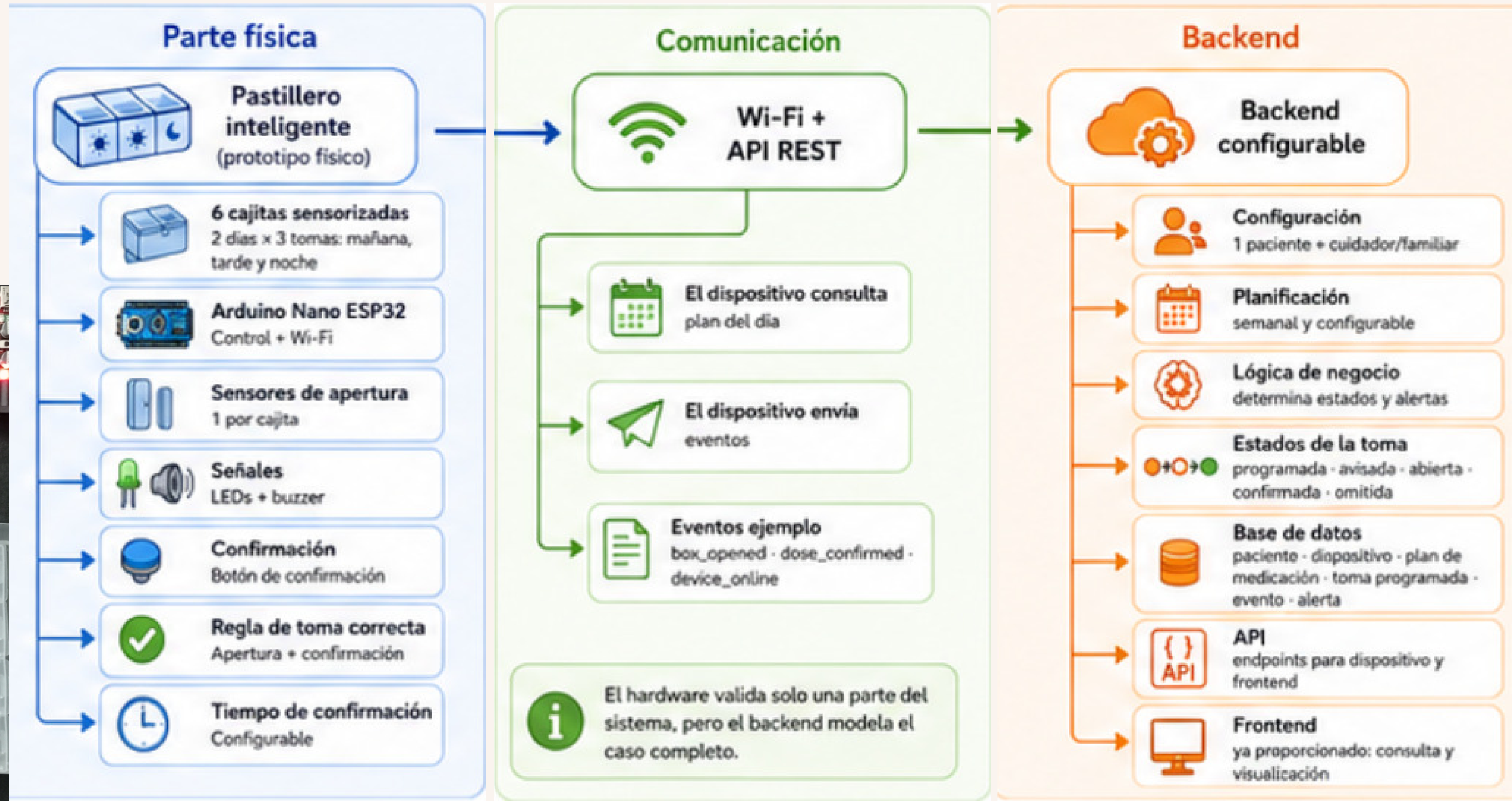
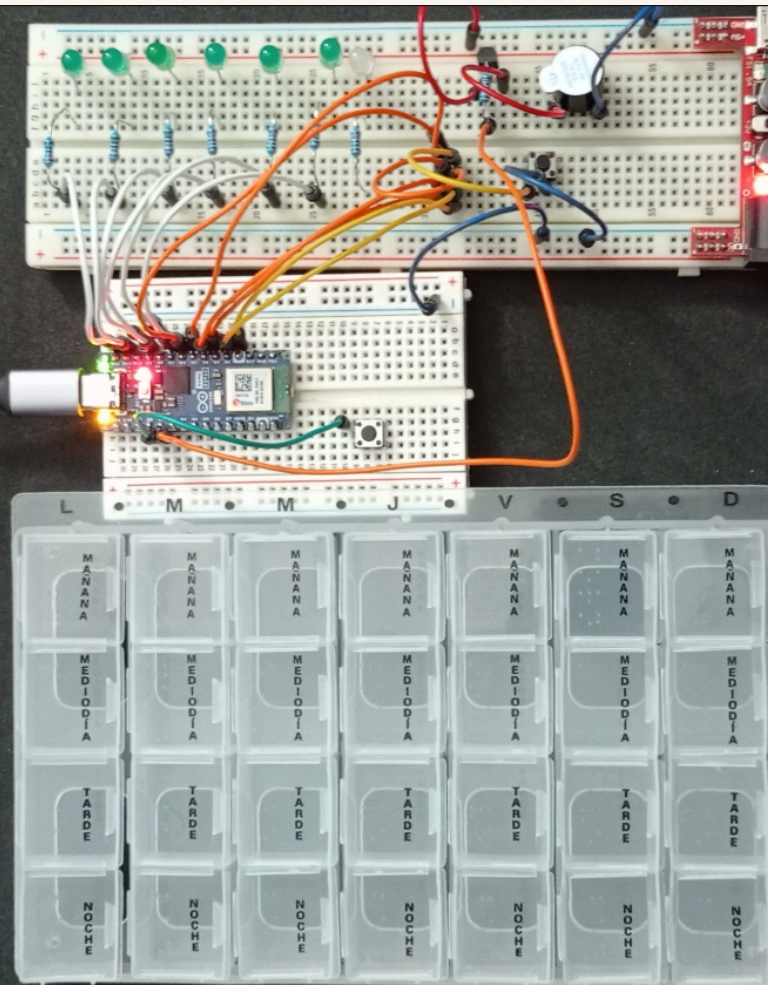
- **Aprendizaje Basado en el Servicio** porque integra aprendizaje académico con servicio comunitario.
- No se hacen todas las fases. Se parte de que ya tenemos recogidas las necesidades del cliente del proyecto (la residencia de mayores).
- Desarrollo incremental por etapas y con entregables parciales.

Proyecto – Grupos de Trabajo

24 Alumnos y Trabajo cooperativo
de 3



Proyecto – Mapa Conceptual



Proyecto – Temporalización

Ubicación : 2º Trimestre. – 14 sesiones

Se integra como bloque específico dentro de la programación general del módulo.

El ritmo previsto es de 2 sesiones semanales :

Sesiones 1-4

Hito 1: Análisis funcional, modelo de dominio y diseño de API REST.

Sesiones 5-8

Hito 2: Arquitectura backend, entorno, persistencia y planificación diaria.

Sesiones 9-11

Hito 3: Registro de eventos, estados de toma, alertas y dashboard.

Sesiones 12-14

Hito 4: Integración, pruebas, documentación técnica y defensa oral.

Proyecto – Secuenciación Visual de Sesiones

	Semana 1 12–16 enero	Semana 2 19–23 enero	Semana 3 26–30 enero	Semana 4 2–6 febrero	Semana 5 9–13 febrero	Semana 6 16–20 febrero	Semana 7 23–27 febrero
Sesión 1 · Presentación del reto	12 enero						
Sesión 2 · Análisis funcional	15 enero						
Sesión 3 · Modelo de dominio		19 enero					
Sesión 4 · Diseño de API REST		22 enero					
Sesión 5 · Arquitectura del backend			26 enero				
Sesión 6 · Puesta en marcha del entorno			29 enero				
Sesión 7 · Persistencia con Prisma/SQLite				2 febrero			
Sesión 8 · Planificación diaria				5 febrero			
Sesión 9 · Registro de eventos					9 febrero		
Sesión 10 · Estados de toma					12 febrero		
Sesión 11 · Alertas y dashboard						16 febrero	
Sesión 12 · Integración y pruebas						19 febrero	
Sesión 13 · Documentación técnica							23 febrero
Sesión 14 · Defensa oral y cierre							26 febrero

Proyecto – Detalle de Sesiones 1 a 5

SES.	Foco	Qué se va a realizar	Materiales / software	Evidencia evaluable
1	Presentación del reto	Presentar el problema, explicar el proyecto, revisar el repositorio y organizar equipos de trabajo.	Aula informática, PCs, proyector, README, GitHub, VS Code.	Registro inicial de comprensión del reto y organización del equipo.
2	Análisis funcional	Identificar actores, eventos, estados y regla de negocio principal.	Plantilla de análisis, documentación del proyecto, editor Markdown.	Documento de análisis funcional: actores, eventos, estados y reglas de negocio.
3	Modelo de dominio	Definir entidades, relaciones y estructura inicial de datos.	Plantilla de modelo, draw.io o similar, documentación técnica.	Modelo de dominio o esquema inicial de datos entregado.
4	Diseño de API	Diseñar endpoints, bodies, respuestas y errores básicos.	Guía API, VS Code, documentación de endpoints.	Diseño documentado de la API: endpoints y peticiones
5	Arquitectura del backend	Revisar la estructura del proyecto, organizar carpetas, rutas, servicios y repositorios.	Node.js, npm, VS Code, Git, terminal.	Repositorio con estructura inicial del backend preparada.

Proyecto – Detalle de Sesiones 6 a 10				
Ses.	Foco	Qué se va a realizar	Materiales / software	Evidencia evaluable
6	Puesta en marcha del entorno	Configurar dependencias, scripts y primera ejecución del backend en local.	Node.js, npm, VS Code, terminal, GitHub.	Entorno local operativo y backend ejecutándose correctamente.
7	Persistencia con Prisma/SQLite	Revisar el esquema de datos, configurar persistencia y comprobar acceso a base de datos.	Prisma, SQLite, VS Code, terminal, cliente HTTP.	Conexión funcional con la base de datos y persistencia configurada.
8	Planificación diaria	Implementar o revisar la consulta del plan del día y su respuesta desde la API.	Backend, Prisma, cliente HTTP, documentación de endpoints.	Consulta funcional de planificación diaria operativo.
9	Registro de eventos	Implementar o revisar la recepción y almacenamiento de eventos desde simulación o dispositivo.	Postman, backend en ejecución, ejemplos JSON.	Endpoint de registro de eventos implementado y probado
10	Estados de toma	Aplicar la lógica de negocio relacionada con apertura, confirmación y estado de la toma.	Casos de prueba, VS Code, Postman, base de datos.	Funcionalidad implementada para actualizar estados de toma y gestionar información persistente.

Proyecto – Detalle de Sesiones 11 a 14

Ses.	Foco	Qué se va a realizar	Materiales / software	Evidencia evaluable
11	Alertas y dashboard	Generar alertas, consultar incidencias y revisar la visualización del seguimiento.	Casos de prueba, backend, cliente HTTP, dashboard o interfaz de consulta.	Alertas generadas y evidencias de seguimiento
12	Integración y pruebas	Comprobar el flujo completo con simulación o hardware, revisar errores y registrar incidencias.	Prototipo o simulación, Postman, terminal, Arduino IDE si procede.	Prueba funcional del flujo completo e informe breve de incidencias.
13	Documentación técnica	Revisar repositorio, ordenar archivos y completar documentación técnica del proyecto.	Repositorio, plantilla documental, editor Markdown.	Código del repositorio probado y documentado, con documentación técnica revisada.
14	Defensa oral y cierre	Preparar y realizar la presentación final del proyecto y la demostración general.	Presentación, repositorio final, herramienta de presentaciones.	Presentación oral, demostración funcional y entrega final del proyecto.

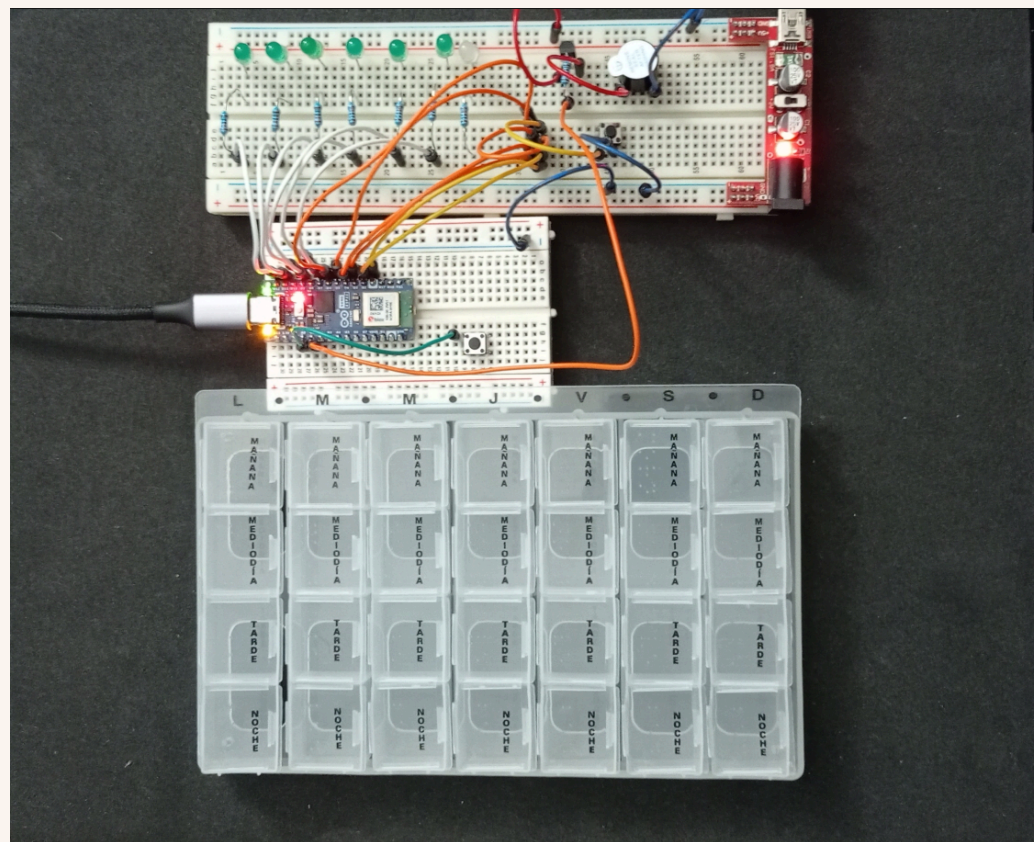
Proyecto – Solución Técnica y Arquitectura

Arduino Nano
ESP32 /
Simulación

API REST · Express
+ TypeScript

Servicios y lógica de
negocio

Persistencia ·
Prisma + SQLite



Stack Tecnológico

Node.js · Express · TypeScript · Prisma · SQLite.
Postman para pruebas.
Jest y Supertest para validación automática

Proyecto – Evaluación

La evaluación es continua y formativa, y se apoya en evidencias observables a lo largo del proyecto.

Centrada en RA5, RA6 y RA7.

La evaluación parte de los RA seleccionados y de sus criterios de evaluación.

Cada criterio se vincula con una evidencia evaluable del proyecto: una actividad, una prueba funcional, una entrega o una defensa y la evidencia se califica mediante un instrumento concreto.

Instrumentos

- Rúbricas de actividades prácticas.
- Lista de control del repositorio.
- Pruebas funcionales del backend.
- Rúbrica documental.
- Rúbrica de defensa oral.



Proyecto – Diseño Universal para el Aprendizaje

- Acceso claro a la información: guías, repositorio y materiales de apoyo.
 - Aprendizaje progresivo: fases, hitos y seguimiento continuo.
 - Participación diversa: trabajo cooperativo y tareas variadas.
 - Apoyos flexibles: plantillas, documentación y recursos técnicos.
 - Sentido del proyecto: problema real, útil y conectado con el módulo.
- 

Proyecto – Evidencias, Instrumentos y Ponderación RA/CE

RA	Criterio de evaluación	Peso CE	Evidencia Evaluable	Instrumento	Momento
RA5	5.a	5%	Informe, esquema o justificación de la arquitectura propuesta, diferenciando lógica de negocio y presentación.	Lista de control	Sesiones 1-5
RA5	5.f	10%	Backend organizado por capas, con lógica de negocio separada y estructura de proyecto coherente.	Rúbrica técnica	Sesiones 5-10
RA5	5.h	5%	Código del repositorio probado y documentado, con evidencias de funcionamiento.	Rúbrica documental	Sesiones 13-14
RA6	6.b	10%	Conexión funcional con la base de datos implementada en el backend.	Rúbrica técnica	Sesiones 7-8
RA6	6.c	10%	Consulta funcional de la planificación diaria desde la base de datos.	Prueba funcional	Sesión 8
RA6	6.f	10%	Funcionalidad implementada para actualizar estados de toma y gestionar información persistente.	Rúbrica técnica	Sesiones 10-11
RA6	6.h	5%	Pruebas realizadas y documentación de la parte de persistencia de datos.	Rúbrica documental	Sesiones 12-13
RA7	7.d	20%	Endpoints implementados para planificación, eventos, alertas y consultas.	Rúbrica de API	Sesiones 4-11
RA7	7.e	10%	Descripción documentada de la API: endpoints, parámetros, cuerpos, respuestas y errores.	Rúbrica documental	Sesiones 4-13
RA7	7.f	10%	Resultados de la verificación de endpoints mediante cliente HTTP y/o pruebas automatizadas.	Prueba funcional	Sesiones 9-12
RA7	7.g	5%	Evidencia de consumo de la API desde simulador, cliente HTTP o interfaz de consulta.	Lista de control	Sesión 12

Proyecto – Repositorio , Materiales y Transferibilidad

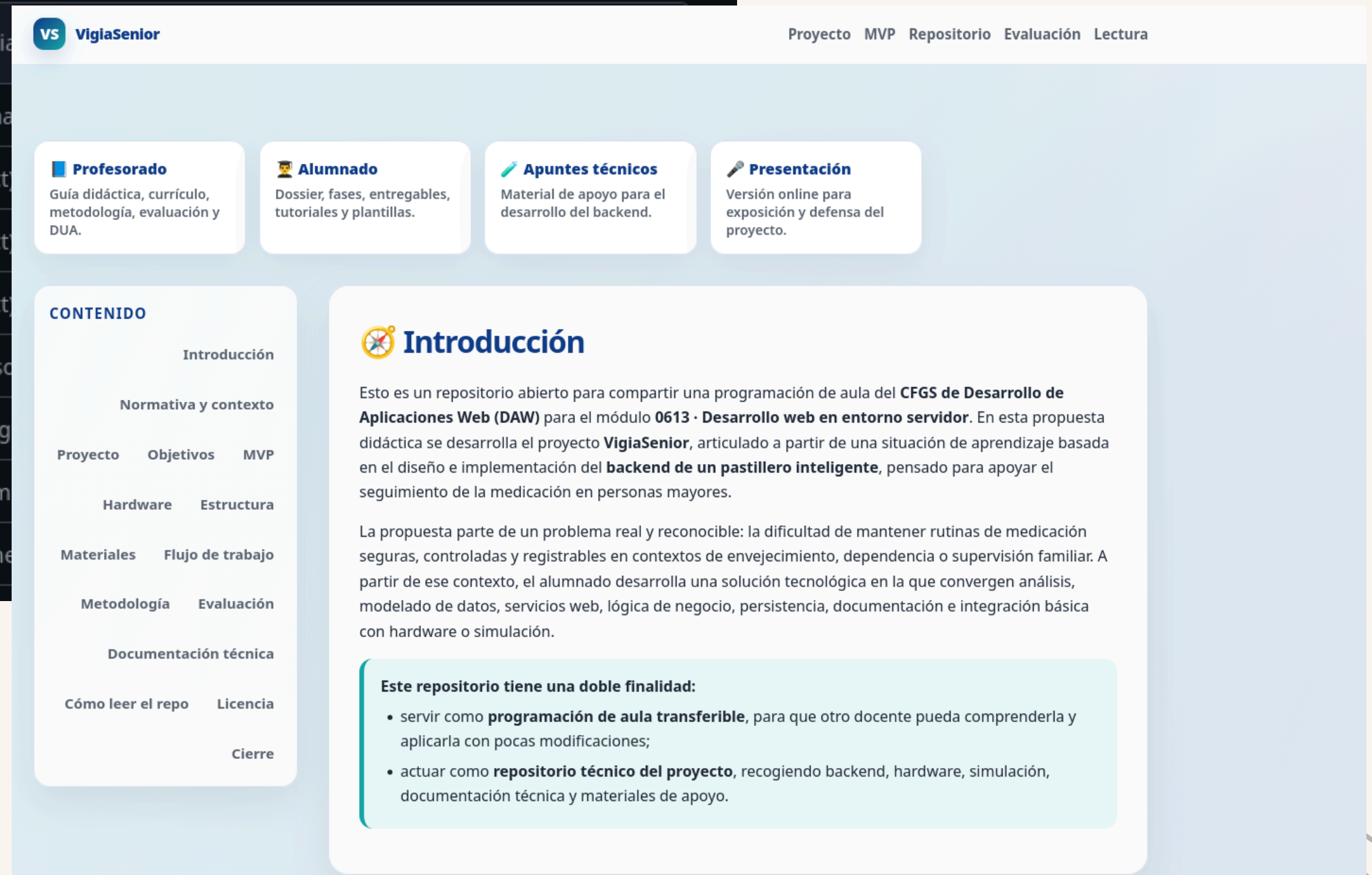
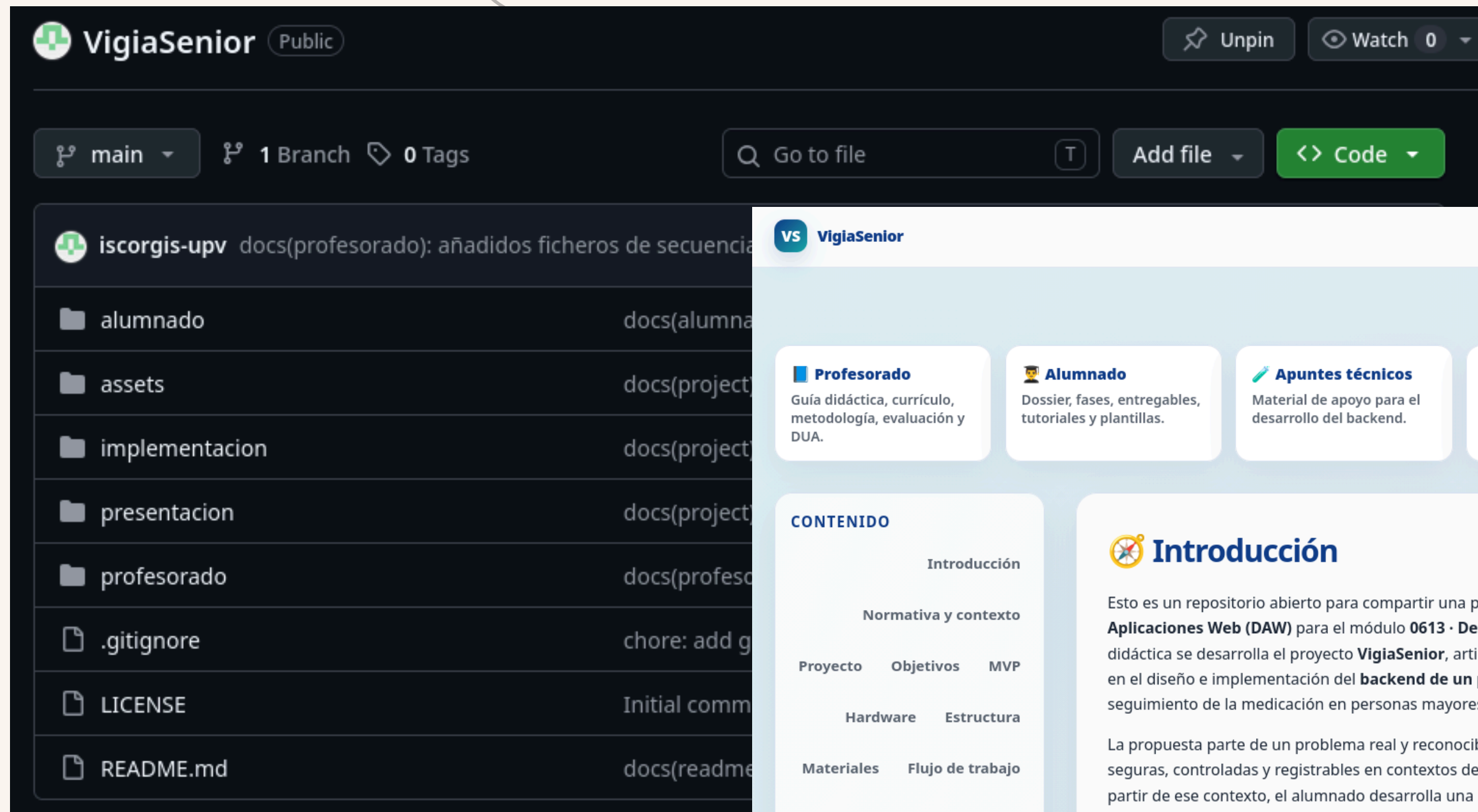
Qué incluye el repositorio:

Programación de aula y documentos curriculares.
Backend y documentación de implementación.
Materiales para profesorado y alumnado.
Recursos de hardware y simulación

Valor didáctico

Facilita que otro docente comprenda y reutilice la propuesta.
Organiza materiales técnicos y pedagógicos en una estructura clara.
Permite trazabilidad del proceso mediante commits y documentación.

Proyecto – Repositorio en Github



GithubPages - VigiaSenior



¡Gracias!